

```
import kagglehub

# Download latest version
path = kagglehub.dataset_download("oxcdcd/cifar10")

print("Path to dataset files:", path)
```

Downloading to /root/.cache/kagglehub/datasets/oxcdcd/cifar10/1.archive...
 100%|██████████| 175M/175M [00:09<00:00, 19.7MB/s]Extracting files...

Path to dataset files: /root/.cache/kagglehub/datasets/oxcdcd/cifar10/versions/1

```
! cp -rf /root/.cache/kagglehub/datasets/oxcdcd/cifar10/versions/1/* .
```

```
import tensorflow as tf
import matplotlib.pyplot as plt
```

```
IMG_SIZE = (224, 224)
BATCH_SIZE = 32
```

```
train_path = "cifar10/train"
test_path = "cifar10/test"
```

```
train_ds = tf.keras.utils.image_dataset_from_directory(
    train_path,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    shuffle=True
)
```

```
test_ds = tf.keras.utils.image_dataset_from_directory(
    test_path,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    shuffle=False
)
```

```
# Removed .range(100) as it replaces image data with scalar indices.
# If you want to limit the dataset size, use .take(100) instead:
```

Found 50000 files belonging to 10 classes.
 Found 10000 files belonging to 10 classes.

```
class_names = train_ds.class_names

plt.figure(figsize=(10,10))

for images, labels in train_ds.take(1):
    for i in range(9):
        plt.subplot(3,3,i+1)
        plt.imshow(images[i].numpy().astype("uint8"))
        plt.title(class_names[labels[i]])
        plt.axis("off")

plt.show()
```

frog



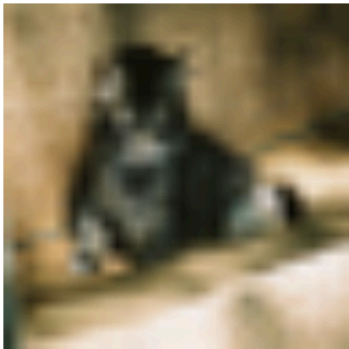
airplane



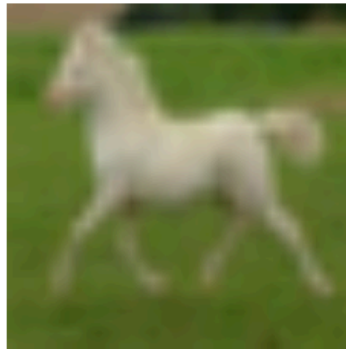
airplane



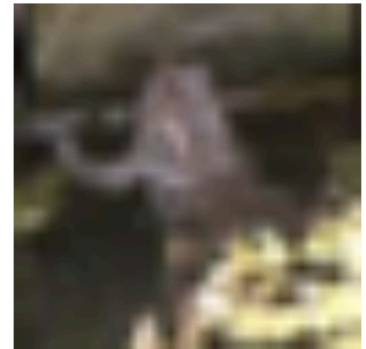
cat



horse



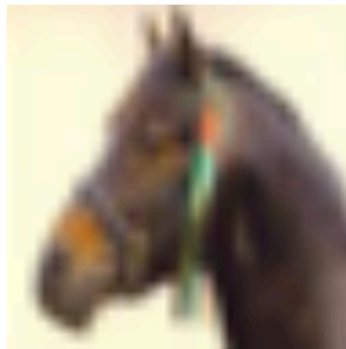
frog



automobile



horse



automobile



```

from tensorflow.keras.applications import VGG16
from tensorflow.keras.layers import GlobalAveragePooling2D, Dense, Dropout
from tensorflow.keras.models import Model

base_model = VGG16(
    weights='imagenet',
    include_top=False,
    input_shape=(224,224,3)
)

base_model.trainable = False

x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(256, activation='relu')(x)
x = Dropout(0.5)(x)

outputs = Dense(len(class_names), activation='softmax')(x)

model = Model(inputs=base_model.input, outputs=outputs)

```

```

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
model.summary()

```

Downloading data from <https://storage.googleapis.com/tensorflow/keras-applications/vgg16/v58889256/58889256> ————— 4s 0us/step

Model: "functional"

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 224, 224, 3)	0
block1_conv1 (Conv2D)	(None, 224, 224, 64)	1,792
block1_conv2 (Conv2D)	(None, 224, 224, 64)	36,928
block1_pool1 (MaxPooling2D)	(None, 112, 112, 64)	0
block2_conv1 (Conv2D)	(None, 112, 112, 128)	73,856
block2_conv2 (Conv2D)	(None, 112, 112, 128)	147,584
block2_pool1 (MaxPooling2D)	(None, 56, 56, 128)	0
block3_conv1 (Conv2D)	(None, 56, 56, 256)	295,168
block3_conv2 (Conv2D)	(None, 56, 56, 256)	590,080
block3_conv3 (Conv2D)	(None, 56, 56, 256)	590,080
block3_pool1 (MaxPooling2D)	(None, 28, 28, 256)	0
block4_conv1 (Conv2D)	(None, 28, 28, 512)	1,180,160
block4_conv2 (Conv2D)	(None, 28, 28, 512)	2,359,808
block4_conv3 (Conv2D)	(None, 28, 28, 512)	2,359,808
block4_pool1 (MaxPooling2D)	(None, 14, 14, 512)	0
block5_conv1 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_conv2 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_conv3 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_pool1 (MaxPooling2D)	(None, 7, 7, 512)	0
global_average_pooling2d (GlobalAveragePooling2D)	(None, 512)	0
dense (Dense)	(None, 256)	131,328
dropout (Dropout)	(None, 256)	0
dense_1 (Dense)	(None, 10)	2,570

Total params: 14,848,586 (56.64 MB)

```

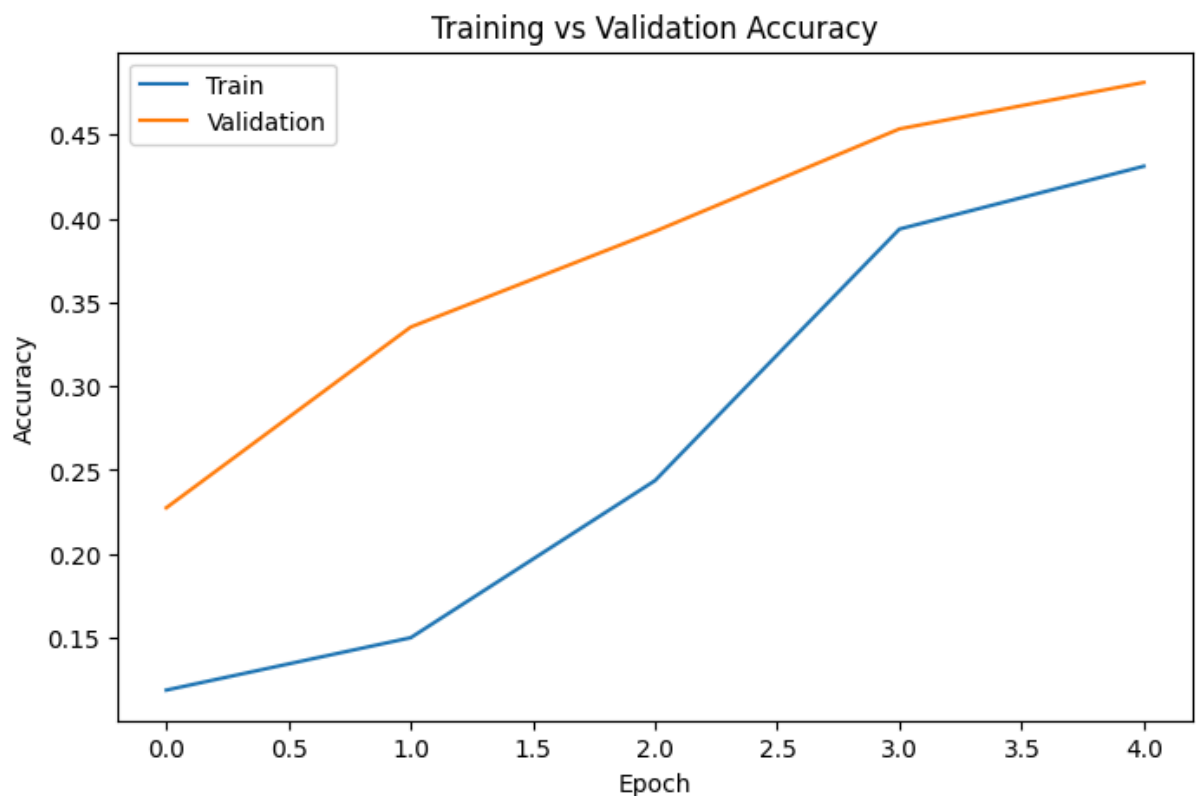
train_ds = train_ds.take(5)
history = model.fit(
    train_ds,
    validation_data=test_ds,

```

```
epochs=5,  
batch_size=4  
)
```

```
Epoch 1/5  
5/5 ————— 76s 15s/step - accuracy: 0.1187 - loss: 6.6075 - val_accuracy: 0.  
Epoch 2/5  
5/5 ————— 56s 14s/step - accuracy: 0.1500 - loss: 4.9143 - val_accuracy: 0.  
Epoch 3/5  
5/5 ————— 55s 14s/step - accuracy: 0.2438 - loss: 3.8250 - val_accuracy: 0.  
Epoch 4/5  
5/5 ————— 55s 14s/step - accuracy: 0.3938 - loss: 2.5808 - val_accuracy: 0.  
Epoch 5/5  
5/5 ————— 55s 14s/step - accuracy: 0.4313 - loss: 1.8123 - val_accuracy: 0.
```

```
plt.figure(figsize=(8,5))  
  
plt.plot(history.history['accuracy'],label='Train')  
  
plt.plot(history.history['val_accuracy'],label='Validation')  
  
plt.xlabel("Epoch")  
  
plt.ylabel("Accuracy")  
  
plt.title("Training vs Validation Accuracy")  
  
plt.legend()  
  
plt.show()
```



Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.